

QazconHub — Детальное документ-описание проекта

Полное техническое описание цифровой платформы логистического терминала ТОО «QazconHub». Документ охватывает архитектуру, стек технологий, схему базы данных, все API-эндпоинты, ролевую модель, структуру фронтенда, PWA-возможности, безопасность и развёртывание.

1. Обзор проекта

QazconHub — это современное **PWA-приложение** (Progressive Web App) для комплексного управления логистическим терминалом. Платформа автоматизирует ключевые операционные процессы терминала:

- 📦 **Контейнерные операции** — учёт, размещение и отслеживание контейнеров.
- 🚂 **Железнодорожные пути (ППЖТ)** — управление путями и вагонами.
- 🏗️ **Спецтехника** — учёт кранов, погрузчиков и тягачей.
- 🚚 **Транспорт и пропускной режим** — въезд/выезд, OCR-распознавание номеров, шлагбаумы, пропуска.
- 👤 **Электронная очередь** — запись водителей и назначение слотов.
- 📄 **Заявки клиентов** — создание и обработка заявок.
- 📖 **Журнал действий и отчёты** — аудит и PDF-отчёты.

Ключевая особенность — **единая роль-ориентированная панель управления**: один и тот же интерфейс автоматически адаптируется под права конкретного пользователя (13 ролей), показывая только доступные ему разделы.

2. Архитектура

Платформа построена по классической **клиент-серверной** модели с **монолитным** бэкендом и **SPA-подобным** фронтендом.



Поток запроса

- Пользователь входит через `POST /api/auth` → получает **JWT-токен** (7 дней).
- Токен сохраняется в `localStorage` (`token`, `user`) и передаётся в заголовке `Authorization: Bearer <token>`.
- Каждый защищённый эндпоинт проходит через middleware `authRequired` (проверка JWT) и, при необходимости, `roleRequired(...)` (проверка роли).

4. Сервер выполняет операции с SQLite и, для изменяющих операций, рассылает **realtime-события** через Socket.IO всем подключённым клиентам в комнате `terminal`.
5. Фронтенд получает событие и автоматически перерисовывает текущий раздел (если он открыт).

3. Технологический стек

Слой	Технология	Назначение
Бэкенд	Node.js (≥18), Express 4	HTTP-сервер, REST API
База данных	SQLite + better-sqlite3	Локальная файловая БД (WAL, FK)
Авторизация	jsonwebtoken (JWT), bcryptjs	Токены и хэширование паролей
Реальное время	Socket.IO 4	Живые обновления интерфейса
Защита	express-rate-limit	Ограничение частоты запросов
Загрузка файлов	multer 2	Загрузка фото (до 5 МБ, изображения)
PDF-отчёты	PDFKit	Генерация отчётов по контейнерам/вагонам
Файловая система	fs-extra	Создание директории загрузок
Фронтенд	HTML5, CSS3, Vanilla JS	Без фреймворков, лёгкий и быстрый
PWA	Service Worker, Web App Manifest	Офлайн-кэш, установка на устройство
Деплой	Docker, Fly.io	Контейнеризация и облачный хостинг

4. Структура проекта

```
delivery-pwa/                                # корень проекта (QazconHub Terminal)
├── server/
│   ├── server.js                            # API-сервер (Express + Socket.IO), ~496 строк
│   ├── db.js                               # схема БД + начальные данные (seed)
│   └── db/
│       └── terminal.db                     # файл базы данных SQLite (+ WAL/SHM)
├── public/
│   ├── index.html                          # страница входа
│   ├── dashboard.html                     # роль-ориентированная панель управления
│   ├── manifest.json                      # PWA-манифест
│   ├── sw.js                              # Service Worker (кэш v2)
│   ├── icon.png                           # иконка приложения
│   └── css/
│       └── style.css                       # стили (адаптивные, 5 брейкпоинтов)
│   └── js/
│       ├── api.js                         # модуль API, авторизация, утилиты, словари
│       ├── dashboard.js                   # ядро панели + модули (часть 1)
│       └── dashboard-views.js             # модули (часть 2) + инициализация
├── uploads/                               # загруженные фото (создаётся автоматически)
├── package.json                           # зависимости и скрипты
├── package-lock.json
├── Dockerfile                             # контейнеризация
├── fly.toml                               # конфигурация Fly.io
├── .env.example                           # шаблон переменных окружения
├── .gitignore
├── README.md                             # краткое описание и быстрый старт
└── PROJECT.md                             # настоящий документ
```

5. База данных (SQLite)

Файл БД: `server/db/terminal.db` . Включены режимы **WAL** (`journal_mode = WAL`) и **внешние ключи** (`foreign_keys = ON`).

5.1 Таблицы

 **users** — пользователи и роли

Поле	Тип	Описание
<code>id</code>	INTEGER PK	Идентификатор
<code>login</code>	TEXT UNIQUE	Логин
<code>pass_hash</code>	TEXT	Хэш пароля (bcrypt)
<code>name</code>	TEXT	ФИО / название
<code>role</code>	TEXT	Роль (13 значений)
<code>terminal</code>	TEXT	Терминал (по умолч. «Основной»)
<code>active</code>	INTEGER	Активен (1/0)
<code>created_at</code>	TEXT	Дата создания

 **containers** — контейнеры

Поле	Тип	Описание
<code>id</code>	INTEGER PK	Идентификатор
<code>number</code>	TEXT UNIQUE	Номер контейнера (напр. TCLU1234567)
<code>size</code>	TEXT	Размер (20/40/45 фт)
<code>type</code>	TEXT	Тип (Гружёный/Порожний/Рефрижератор)
<code>status</code>	TEXT	Статус (см. словарь)
<code>zone / row / stack / tier</code>	TEXT	Размещение: зона/ряд/ярус/уровень
<code>client</code>	TEXT	Клиент
<code>cargo</code>	TEXT	Груз
<code>weight</code>	REAL	Вес (т)
<code>created_at / updated_at</code>	TEXT	Метки времени

 **tracks** — железнодорожные пути

Поле	Тип	Описание
<code>id</code>	INTEGER PK	Идентификатор
<code>name</code>	TEXT UNIQUE	Название (Путь №1...)
<code>capacity</code>	INTEGER	Вместимость (вагонов)
<code>zone</code>	TEXT	Зона
<code>status</code>	TEXT	Статус (свободен)

 **wagons** — вагоны

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
number	TEXT UNIQUE	Номер вагона
cargo	TEXT	Груз
owner	TEXT	Владелец
track_id	INTEGER FK→tracks	Путь
status	TEXT	Статус (см. словарь)
direction	TEXT	Направление
dest_station	TEXT	Станция назначения
shipper	TEXT	Отправитель
operation	TEXT	Операция (прибытие/выгрузка/...)
created_at / updated_at	TEXT	Метки времени

 equipment — спецтехника

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
name	TEXT	Название (Кран КК-1...)
type	TEXT	Тип (Козловой кран и т.д.)
plate	TEXT	Госномер
status	TEXT	Статус (свободна/работает/на ремонте)
driver	TEXT	Водитель
lat / lon	REAL	Координаты (GPS)
fuel	REAL	Топливо (%)
hours	REAL	Моточасы
created_at	TEXT	Дата создания

 vehicles — транспорт (въезд/выезд)

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
plate	TEXT	Госномер
type	TEXT	Тип (Грузовой/Легковой/...)
driver	TEXT	Водитель
purpose	TEXT	Цель визита
direction	TEXT	Направление (въезд)
status	TEXT	Статус (на_территории/выехал)
time_in	TEXT	Время въезда
time_out	TEXT	Время выезда
container_id	INTEGER FK→containers	Связанный контейнер
created_at	TEXT	Дата создания

 requests — заявки клиентов

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
client_id	INTEGER FK→users	Клиент
client_name	TEXT	Имя клиента
type	TEXT	Тип (контейнер/вагон/хранение/другое)
container_number	TEXT	Номер контейнера
cargo	TEXT	Груз
weight	REAL	Вес (т)
status	TEXT	Статус (см. словарь)
date_slot / time_slot	TEXT	Желаемый слот
comment	TEXT	Комментарий
created_at / updated_at	TEXT	Метки времени

 queue — электронная очередь

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
plate	TEXT	Госномер
driver	TEXT	Водитель
purpose	TEXT	Цель
slot	TEXT	Назначенный слот
status	TEXT	Статус (см. словарь)
created_at	TEXT	Дата создания

 passes — пропуска

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
code	TEXT UNIQUE	Код пропуска (PASS-...)
vehicle_id	INTEGER FK→vehicles	Транспорт
plate	TEXT	Госномер
type	TEXT	Тип (разовый/постоянный)
status	TEXT	Статус (активен/использован)
created_at	TEXT	Дата создания

 audit_log — журнал действий

Поле	Тип	Описание
id	INTEGER PK	Идентификатор
user_id	INTEGER	Пользователь

Поле	Тип	Описание
<code>user_name</code>	TEXT	Имя пользователя
<code>action</code>	TEXT	Код действия (container:create и т.д.)
<code>details</code>	TEXT	Детали
<code>created_at</code>	TEXT	Дата создания

5.2 Начальные данные (seed)

При первом запуске `db.js` автоматически создаёт (только если таблицы пусты):

- **13 пользователей** (см. таблицу ролей в разделе 7).
- **4 железнодорожных пути**: Путь №1–2 (Зона А, 25 ваг.), Путь №3–4 (Зона Б, 20 ваг.).
- **4 единицы спецтехники**: Кран КК-1, Кран КК-2, Погрузчик П-1, Тягач Т-1.

6. Словари статусов

Статусы хранятся в БД в **машинном** виде (латиницей, нижний регистр) и отображаются пользователю в **человекочитаемом** виде через словари в `api.js`.

Статусы контейнеров (`CONTAINER_STATUS`)

Код	Отображение
<code>на_терминале</code>	На терминале
<code>ожидает_вывоза</code>	Ожидает вывоза
<code>вывезен</code>	Вывезен
<code>в_обработке</code>	В обработке

Статусы вагонов (`WAGON_STATUS`)

Код	Отображение
<code>на_пути</code>	На пути
<code>выгрузка</code>	Выгрузка
<code>погрузка</code>	Погрузка
<code>готов_к_отправке</code>	Готов к отправке
<code>отправлен</code>	Отправлен

Статусы заявок (`REQUEST_STATUS`)

Код	Отображение
<code>новая</code>	Новая
<code>подтверждена</code>	Подтверждена
<code>выполняется</code>	Выполняется
<code>завершена</code>	Завершена
<code>отклонена</code>	Отклонена



Статусы очереди (`QUEUE_STATUS`)

Код	Отображение
<code>ожидание</code>	Ожидание
<code>вызван</code>	Вызван
<code>на_территории</code>	На территории
<code>завершён</code>	Завершён



Статусы спецтехники (`EQUIPMENT_STATUS`)

Код	Отображение
<code>свободна</code>	Свободна
<code>работает</code>	Работает
<code>на_ремонте</code>	На ремонте



Статусы транспорта (`VEHICLE_STATUS`)

Код	Отображение
<code>на_территории</code>	На территории
<code>выехал</code>	Выехал

Каждому статусу соответствует CSS-класс бейджа (через `statusClass()`): `st-ok`, `st-done`, `st-warn`, `st-info`, `st-new`, `st-assigned`, `st-onway`, `st-cancelled`.

7. Роли и права доступа

7.1 Список ролей (13)

Роль (код)	Отображение	Логин	Пароль
<code>admin</code>	Администратор	<code>admin</code>	<code>admin123</code>
<code>director</code>	Директор	<code>director</code>	<code>director123</code>
<code>dispatcher</code>	Диспетчер	<code>dispatcher</code>	<code>disp123</code>
<code>ppjt</code>	Диспетчер ППЖТ	<code>ppjt</code>	<code>ppjt123</code>
<code>receiver</code>	Приёмосдатчик	<code>receiver</code>	<code>recv123</code>
<code>crane</code>	Крановщик	<code>crane</code>	<code>crane123</code>
<code>store</code>	Кладовщик	<code>store</code>	<code>store123</code>
<code>guard</code>	Охрана	<code>guard</code>	<code>guard123</code>
<code>customs</code>	Таможенный отдел	<code>customs</code>	<code>customs123</code>
<code>finance</code>	Расчётный отдел	<code>finance</code>	<code>fin123</code>
<code>shift</code>	Начальник смены	<code>shift</code>	<code>shift123</code>
<code>client</code>	Клиент	<code>client</code>	<code>client123</code>
<code>driver</code>	Водитель	<code>driver</code>	<code>driver123</code>

7.2 Доступ к разделам панели (матрица NAV)

Матрица задана в `dashboard.js` (объект `NAV`). Каждая роль видит только свои разделы.

Раздел	admin	director	dispatcher	ppjt	receiver	crane	store	guard	customs	finance	shift	client	driver
 Обзор	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	—	—
 Контейнеры	✓	—	✓	—	✓	✓	✓	—	✓	—	✓	—	—
 Пути и вагоны	✓	—	—	✓	✓	—	—	—	—	—	✓	—	—
 Спецтехника	✓	—	✓	—	—	✓	—	—	—	—	✓	—	—
 Транспорт	✓	—	✓	—	—	—	—	✓	—	—	✓	—	—
 Заявки	✓	—	✓	—	—	—	—	—	✓	✓	✓	✓	—
 Очередь	✓	—	✓	—	—	—	—	✓	—	—	✓	✓	✓
 Пропускной режим	✓	—	✓	—	—	—	—	✓	—	—	✓	—	—
 Журнал действий	✓	✓	—	—	—	—	—	—	—	—	✓	—	—
 Отчёты	✓	✓	—	—	—	—	—	—	—	✓	✓	—	—

7.3 Права на операции (RBAC на сервере)

Права на создание/редактирование проверяются на сервере через `roleRequired(...)`:

Ресурс	Создание	Редактирование
Контейнеры	admin, dispatcher, receiver, guard, shift	admin, dispatcher, receiver, crane, shift
Пути	admin, ppjt	—
Вагоны	admin, ppjt, receiver	admin, ppjt, receiver, shift
Спецтехника	admin, dispatcher, shift	admin, dispatcher, crane, shift
Транспорт (въезд)	admin, guard, dispatcher, shift	admin, guard, dispatcher, shift (выезд)
Заявки	client, admin, dispatcher	admin, dispatcher, shift, client (только свои)
Очередь	driver, client, admin, dispatcher	admin, dispatcher, guard, shift
Пропуска	admin, guard, dispatcher, shift	—
OCR / шлагбаум	admin, guard, dispatcher, shift	—
Журнал (просмотр)	admin, director, shift	—
Отчёты (PDF)	admin, director, finance, shift	—

8. API-эндпоинты (REST)

Базовый URL: `/api`. Все эндпоинты, кроме `POST /api/auth`, требуют заголовок `Authorization: Bearer <JWT>`.

8.1 Аутентификация и справочники

Метод	Путь	Роли	Описание
POST	/api/auth	— (публичный, rate-limit 30/15мин)	Вход: {login, pass} → {token, user}
GET	/api/roles	все	Справочник ролей (код → название)
GET	/api/stats	все	Сводная статистика терминала

GET /api/stats возвращает: containers, containersFull, wagons, wagonsOnTrack, equipment, eqFree, vehicles, queue, requestsNew.

8.2 Контейнеры

Метод	Путь	Роли	Описание
GET	/api/containers	все	Список контейнеров
POST	/api/containers	admin, dispatcher, receiver, guard, shift	Создать контейнер
PATCH	/api/containers/:id	admin, dispatcher, receiver, crane, shift	Обновить (статус, размещение, клиент, груз, вес)

8.3 Пути (ППЖТ)

Метод	Путь	Роли	Описание
GET	/api/tracks	все	Пути + вагоны на них + заполненность
POST	/api/tracks	admin, ppjt	Создать путь

8.4 Вагоны

Метод	Путь	Роли	Описание
GET	/api/wagons	все	Список вагонов (с названием пути)
POST	/api/wagons	admin, ppjt, receiver	Добавить вагон (прибытие)
PATCH	/api/wagons/:id	admin, ppjt, receiver, shift	Обновить (путь, статус, направление, станция)

8.5 Спецтехника

Метод	Путь	Роли	Описание
GET	/api/equipment	все	Список техники
POST	/api/equipment	admin, dispatcher, shift	Добавить технику
PATCH	/api/equipment/:id	admin, dispatcher, crane, shift	Обновить (статус, водитель, топливо, моточасы, GPS)

8.6 Транспорт

Метод	Путь	Роли	Описание
GET	/api/vehicles	все	Список транспорта
POST	/api/vehicles	admin, guard, dispatcher, shift	Регистрация въезда
PATCH	/api/vehicles/:id/exit	admin, guard, dispatcher, shift	Оформление выезда

8.7 Заявки клиентов

Метод	Путь	Роли	Описание
GET	/api/requests	все	Список (клиент видит только свои)
POST	/api/requests	client, admin, dispatcher	Создать заявку
PATCH	/api/requests/:id	admin, dispatcher, shift, client	Обновить (клиент — только свою)

8.8 Электронная очередь

Метод	Путь	Роли	Описание
GET	/api/queue	все	Список очереди
POST	/api/queue	driver, client, admin, dispatcher	Встать в очередь
PATCH	/api/queue/:id	admin, dispatcher, guard, shift	Обновить (статус, слот)

8.9 Пропуска, OCR, шлагбаумы

Метод	Путь	Роли	Описание
GET	/api/passes	admin, guard, dispatcher, shift	Список пропусков
POST	/api/passes	admin, guard, dispatcher, shift	Создать пропуск (код PASS-...)
POST	/api/ocr/recognize	admin, guard, dispatcher, shift	Распознать номер → доступ/запрет
POST	/api/barrier/:action	admin, guard, dispatcher, shift	Открыть/закрыть шлагбаум (open / close)

8.10 Журнал и отчёты

Метод	Путь	Роли	Описание
GET	/api/audit	admin, director, shift	Журнал действий (последние 200)
GET	/api/report/containers	admin, director, finance, shift	PDF-отчёт по контейнерам
GET	/api/report/wagons	admin, director, finance, shift	PDF-отчёт по вагонам

8.11 Загрузка файлов

Метод	Путь	Роли	Описание
POST	/api/upload-photo	все (авторизованные)	Загрузка фото (multipart, поле photo) → {url}

9. Реальное время (Socket.IO)

Сервер инициализирует Socket.IO поверх HTTP-сервера с CORS `origin: '*'`.

События клиент → сервер

- `terminal:join` — клиент входит в общую комнату `terminal` и, при передаче роли, в комнату `role:<роль>`.

События сервер → клиент (`emitUpdate`)

Сервер рассылает события всем в комнате `terminal`. Фронтенд перерисовывает раздел, только если он сейчас открыт:

Событие	Триггер	Раздел
<code>container:update</code>	создание/изменение контейнера	Контейнеры
<code>wagon:update</code>	добавление/изменение вагона	Пути и вагоны
<code>equipment:update</code>	изменение техники	Спецтехника
<code>vehicle:update</code>	въезд/выезд транспорта	Транспорт
<code>request:update</code>	создание/изменение заявки	Заявки
<code>queue:update</code>	изменение очереди	Очередь
<code>barrier:update</code>	открытие/закрытие шлагбаума	Пропускной режим

10. Фронтенд

10.1 Страница входа — `index.html`

- Форма логина/пароля, отправка на `POST /api/auth`.
- При успехе токен и пользователь сохраняются в `localStorage`, редирект на `/dashboard.html`.
- Если уже авторизован — сразу переход в панель.
- Сетка демо-доступов для быстрого входа.

10.2 Панель управления — `dashboard.html`

Структура:

- **Сайдбар** (`.sidebar`): бренд, навигация (`#sideNav`), блок пользователя с кнопкой выхода.
- **Основная область** (`.main`): топбар (кнопка меню `≡`, заголовок, индикатор «Онлайн», часы) и контент (`#content`).
- **Оверлей** (`.sidebar-overlay`) — для мобильного меню.
- **Модальное окно** (`.modal-overlay` / `.modal`).

10.3 Модуль API — `api.js`

- `API.getToken/setToken/getUser/setUser/logout` — работа с `localStorage`.
- `API.request/get/post/patch` — обёртки над `fetch` с авторизацией и обработкой 401.
- `API.uploadPhoto` — загрузка файла через `FormData`.
- `connectSocket()` — подключение Socket.IO с токеном.
- Словари статусов, `ROLE_LABELS`, `statusLabel`, `statusClass`, `esc` (XSS-защита), `fmtDate`, `toast`.

10.4 Ядро панели — `dashboard.js` (часть 1)

- Проверка авторизации (редирект на `/` при отсутствии).
- Объект `NAV` (10 разделов с ролями) и вычисление `allowedNav` по роли.
- Часы в реальном времени.
- `buildNav()`, `switchView()`, `openModal()`, `closeModal()`, `render()`.
- Модули: **Обзор**, **Контейнеры**, **Пути и вагоны**, **Спецтехника**, **Транспорт**.
- Экспорт ядра в `window.__dashboardCore` (включая `VIEWS`).

10.5 Дополнительные модули — `dashboard-views.js` (часть 2)

- Модули: **Заявки**, **Очередь**, **Пропускной режим** (OCR + шлагбаум), **Журнал действий**, **Отчёты**.
- Инициализация: `buildNav()`, первичный `render()`.
- Подключение Socket.IO и обработка realtime-событий.

- Кнопка выхода.
- **Мобильное меню:** открытие/закрытие с оверлеем, Escape, свайпы, блокировка прокрутки, сброс при resize.

10.6 Стили — `style.css`

- Тёмно-синяя тема (`#0f2a4a` → `#0b2138`), акцент `#ffb300` .
- Сайдбар 250px, сетка статистики `auto-fill minmax(200px, 1fr)` , таблицы с горизонтальным скроллом (`.table-wrap`).
- **Адаптивность — 5 брейкпоинтов:**
 - `≤1100px` — планшет (компактная сетка).
 - `≤900px` — мобильное меню (off-canvas сайдбар 270px, кнопка ≡).
 - `≤600px` — мобильный (одна колонка).
 - `≤480px` — вход (адаптация).
 - `≤380px` — очень маленькие экраны (сайдбар на всю ширину).
- `.grid-2 > * { min-width: 0 }` — защита от горизонтального переполнения таблиц.

11. PWA (Progressive Web App)

11.1 Манифест — `manifest.json`

- `name` : «QazconHub — Логистический терминал», `short_name` : «QazconHub».
- `display: standalone` , `theme_color: #0f2a4a` , `background_color: #f0f4f8` .
- Иконки 192×192 и 512×512 (`icon.png` , `purpose: any maskable`).
- `start_url: /` , `lang: ru` .

11.2 Service Worker — `sw.js`

- Кэш: `qazconhub-cache-v2` (версия обновляется при изменениях статики).
- **Статические ресурсы** кэшируются при установке: `/` , `index.html` , `dashboard.html` , `style.css` , `api.js` , `dashboard.js` , `dashboard-views.js` , `manifest.json` .
- **Активация:** удаляются все старые кэши (кроме текущего), `clients.claim()` .
- **Стратегия кэширования:**
 - **API и Socket.IO** (`/api` , `/socket.io`) — **не кэшируются** (всегда сеть).
 - **Навигация** — `network-first` с фолбэком на кэш (офлайн-доступ к панели).
 - **Статика** — `cache-first` с фоновым обновлением.
- Версия кэша инкрементируется при каждом изменении статики (сейчас `v2`), что форсирует обновление у клиентов.

12. Безопасность

Платформа реализует многоуровневую защиту:

Мера	Реализация
Хэширование паролей	bcrypt (cost 10) — пароли не хранятся в открытом виде
JWT-авторизация	Токены со сроком 7 дней, подпись секретным ключом
Ролевой доступ (RBAC)	<code>roleRequired(...)</code> на каждом изменяющем эндпоинте
Rate limiting	API: 500 запросов/15 мин; вход: 30 попыток/15 мин
XSS-защита	Экранирование вывода через <code>esc()</code> на фронтенде

Мера	Реализация
Аудит действий	Каждое значимое действие пишется в <code>audit_log</code>
Ограничение загрузок	Только изображения, максимум 5 МБ (multer)
Обработка ошибок	Централизованный middleware, без утечки деталей
Секреты в окружении	<code>JWT_SECRET</code> через <code>.env</code> (не в коде)

Ключевые middleware-защиты

- `authRequired` — проверяет заголовок `Authorization: Bearer <token>`, расшифровывает JWT и кладёт `req.user`. При недействительном токене — `401`.
- `roleRequired(...roles)` — проверяет, что роль `req.user.role` входит в разрешённый список. Иначе — `403`.
- `apiLimiter` — глобальный лимит на все `/api`-запросы.
- `authLimiter` — усиленный лимит на эндпоинт входа (защита от brute-force).

13. Развёртывание

13.1 Локальный запуск

```
# Установка зависимостей
npm install

# Настройка окружения (опционально)
cp .env.example .env

# Запуск сервера (порт 8080)
npm start
```

Сервер доступен на `http://localhost:8080`. База данных `server/db/terminal.db` создаётся автоматически при первом запуске вместе с демо-данными.

13.2 Переменные окружения (`.env`)

Переменная	По умолчанию	Описание
<code>JWT_SECRET</code>	<code>qazconhub-terminal-secret-change-in-prod</code>	Секрет подписи JWT (обязательно сменить в проде)
<code>PORT</code>	<code>8080</code>	Порт HTTP-сервера

13.3 Docker

`Dockerfile` собирает образ на базе `node:20-alpine`:

```
docker build -t qazconhub-terminal .
docker run -p 8080:8080 qazconhub-terminal
```

Особенности:

- Устанавливаются инструменты сборки (`python3 make g++`) для нативного модуля `better-sqlite3`.
- `npm install --production` — только продакшен-зависимости.
- Создаётся директория `public/uploads`.
- `NODE_ENV=production`, порт `8080`.

13.4 Fly.io

`fly.toml` — конфигурация облачного деплоя:

```
fly launch
fly deploy
```

Конфигурация: приложение `qazconhub-terminal`, регион `ams`, HTTP-сервис на порту `8080` с `force_https`, авто-стоп/старт машин, 1 виртуальная машина (512 МБ RAM, 1 CPU).

14. Тестирование и проверка

Платформа тестировалась через Playwright (браузерная автоматизация) на трёх размерах экрана:

Размер	Проверка
375px (мобильный)	Сайдбар off-canvas, кнопка ≡, статистика в 1 колонку, таблицы со скроллом, нет горизонтального переполнения
768px (планшет)	Сайдбар off-canvas (270px), кнопка ≡, статистика в 4 колонки
1280px (десктоп)	Сайдбар статичный, кнопка меню скрыта, сетки в несколько колонок

Проверены сценарии:

- Вход под разными ролями (admin — 10 разделов, guard — 4 раздела).
- Открытие/закрытие мобильного меню (кнопка, пункт меню, Escape, оверлей, свайпы).
- Блокировка/разблокировка прокрутки тела при открытом меню.
- Отсутствие ошибок в консоли браузера.

15. Возможности для развития

- Подключение **реального OCR** (вместо симуляции распознавания номера).
- Интеграция с **GPS-трекингом** спецтехники (поля `lat` / `lon` уже есть в схеме).
- **Уведомления** (push) для клиентов и водителей.
- **Мультитерминальная** поддержка (поле `terminal` у пользователей).
- Управление пользователями (CRUD) через админ-панель.
- Расширение отчётности (экспорт в Excel, графики).
- Резервное копирование базы данных.

16. Заключение

QazconHub — полнофункциональная, готовая к эксплуатации PWA-платформа управления логистическим терминалом. Она объединяет в едином адаптивном интерфейсе все ключевые операционные процессы: от учёта контейнеров и вагонов до пропускного режима с OCR и электронной очереди. Благодаря ролевой модели (13 ролей), realtime-синхронизации через Socket.IO, PWA-возможностям (офлайн, установка) и адаптивному дизайну (мобильные/планшетные/десктопные экраны) платформа пригодна для использования как на стационарных рабочих местах терминала, так и в полевых условиях сотрудниками, клиентами и водителями.